

Escola Superior de Tecnologia e Gestão de Águeda
Universidade de Aveiro
Redes e Sistemas Informáticos

Glória de Almeida Lopes Henriques - 120843

Rodrigo de Jesus Marques - 127829

João Gabriel Botelho - 126991

João Almeida - 127140



Mini Projeto
Weather2Travel

Águeda
2025

ÍNDICE

Introdução	4
Diagrama UML.....	5
Imports	6
<i>Import customtkinter</i>	6
<i>Import json</i>	6
<i>From PIL import Image</i>	6
<i>From time import sleep</i>	7
<i>From tkinter import messagebox</i>	7
<i>From datetime import datetime</i>	7
Base de dados	8
<i>Estrutura da base de dados</i>	8
<i>Funcionalidades principais</i>	9
API.....	12
<i>Módulo de integração com a API do IPMA</i>	12
<i>Obter todas as nossas localidades</i>	12
<i>Obter a previsão agregada</i>	12
<i>Obter ID Global de uma Localidade</i>	13
<i>Dicionário e Funções Auxiliares</i>	13
<i>Função filtrar_localidades_por_distrito (localidades, distrito_id)</i>	14
Caixa de texto	15
Módulo auxiliar	17
<i>Classe histórico</i>	17
<i>Classe utilizador</i>	17
<i>Classe remover_frames</i>	19
<i>Classe Esconder_frames</i>	19
<i>Classe visualizar_senha</i>	20
<i>Classe EmailConfirmacao</i>	22
<i>Classe EnviarEmailVerificacao</i>	23
Interface gráfica	25
<i>Classe App</i>	25
<i>Função __init__</i>	25
<i>Função func_frame_boasvindas</i>	26

Função interna func_frame_informacoes.....	27
Model.Esconder_frames.limpar_widgets_do_frame (frame).....	27
Self.dados.caminho_recurso (caminho).....	27
Função func_frame_login.....	27
Função func_frame_criar_conta	28
Carregamento de dados da API e atualização dos menus dropdown	28
Função criar_conta	28
Função mostrar_frame_codigo_confirmacao	30
Função verificar_codigo	31
Função fuc_frame_mail_mudar_senha	32
Função mostrar_frame_codigo	33
Função confirmar_senha	34
Função verificar_login	35
Função destruir_janela	36
Função atualizar_previsao_destino	38
Função mostrar_previsao.....	40
Função atualizar_previsao_localidade	42
Função calcular_media_registos	43
Função ver_media.....	44
Conclusão	46

Introdução

O presente projeto foi desenvolvido no âmbito da unidade curricular de Programação Aplicada, por quatro estudantes do curso de Redes e Sistemas Informáticos da Escola Superior de Tecnologia e Gestão de Águeda – Universidade de Aveiro.

Com este trabalho, desenvolvemos uma aplicação gráfica que recorre à tecnologia RFID (Radio-Frequency Identification) para permitir ao utilizador criar uma conta pessoal, autenticar-se e, posteriormente, aceder a informações meteorológicas personalizadas. O principal objetivo consistiu em associar o código RFID do utilizador à sua localidade Natal, permitindo-lhe consultar facilmente o estado do tempo da sua região de origem, bem como de outras localidades portuguesas à sua escolha.

A aplicação foi concebida com foco na simplicidade e eficiência de utilização. Após um processo de registo que inclui a leitura do UID (Identificador Único) através de um leitor RFID, o utilizador pode efetuar o login utilizando esse identificador ou, em alternativa, métodos tradicionais de autenticação. Uma vez autenticado, tem acesso a uma interface intuitiva que apresenta as previsões meteorológicas da sua localidade Natal e de uma segunda cidade selecionada.

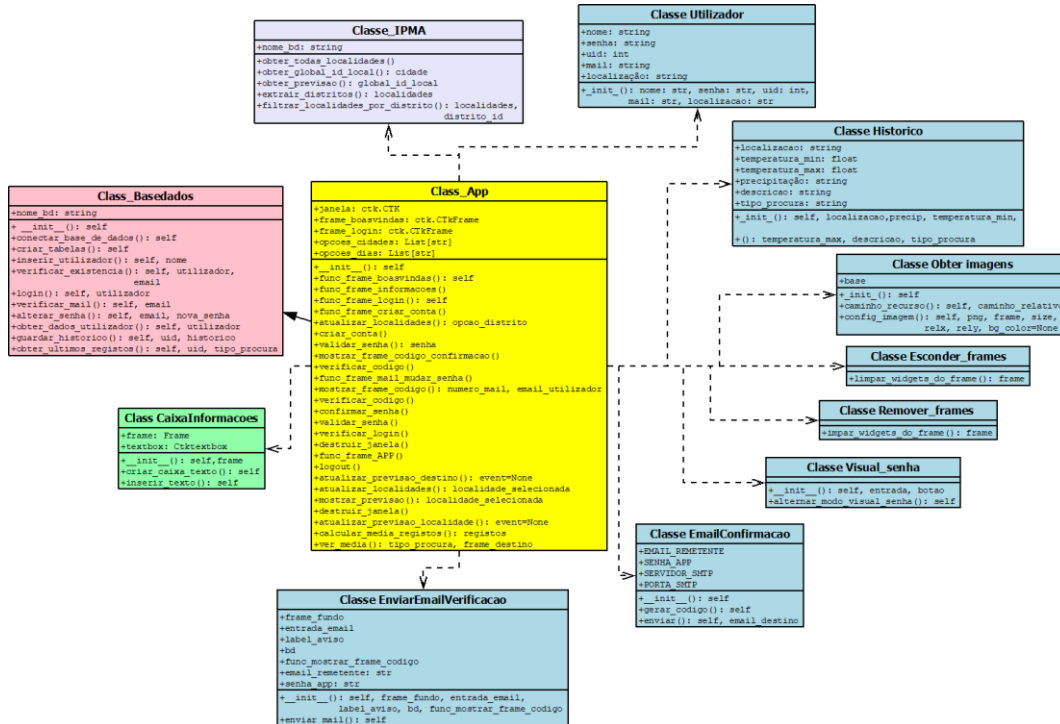
Para garantir a fiabilidade dos dados meteorológicos apresentados, a aplicação recorre a uma API de meteorologia que fornece informações em tempo real, incluindo temperatura máxima e mínima, percentagem de precipitação e data da previsão.

O desenvolvimento foi orientado por uma abordagem de programação orientada a objetos, suportada por um diagrama UML que guiou a estruturação das classes e as suas interações. A aplicação foi implementada em Python, com recurso à biblioteca customtkinter para a construção da interface gráfica, e outras bibliotecas complementares conforme necessário. Foi ainda implementada uma base de dados para gestão dos utilizadores, assegurando a persistência da informação.

Este projeto permitiu aplicar, de forma prática, conceitos de programação orientada a objetos, integração com hardware (leitor RFID) e consumo de serviços web, promovendo a consolidação de competências essenciais para o desenvolvimento de software com interfaces gráficas e tratamento de dados em tempo real, com especial atenção à usabilidade e segurança da aplicação.

Diagrama UML

Na imagem abaixo está demonstrado como nós pensamos inicialmente como fazer o nosso projeto em conjunto.



Imports

O comando **import** é uma instrução que permite utilizar funcionalidades de módulos ou bibliotecas no código principal, neste caso.

```
import customtkinter as ctk
import json
from PIL import Image
from time import sleep
from Base_de_Dados import BaseDados
from tkinter import messagebox
from API import classe_ipma, distritos_por_id, filtrar_localidades_por_distrito, extrair_distritos
from datetime import datetime
from caixa_de_texto import CaixaInformacoes
from model import Historico, Utilizador, Obter_imagens, Remover_frames, Visual_senha, EmailConfirmacao, EnviarEmailVerificacao
```

Import customtkinter

```
import customtkinter as ctk
```

A biblioteca **customtkinter** é uma versão personalizada do **tkinter**, que é uma versão que tem uma aparência mais atualizada e estilizada para interfaces gráficas. Vai permitir que criemos a nossa aplicação com um design mais atrativo e com componentes que podem ser facilmente personalizados. Foi usado maioritariamente na interface gráfica.

Import json

```
import json
```

JSON (JavaScript Object Notation) é um formato de texto que é fácil de ler e escrever para nós, humanos, e é fácil de analisar e gerar para máquinas. Neste caso nosso projeto é usado como um ficheiro de suporte para a coluna **Descricao** na tabela **Historico** do ficheiro **Base_de_Dados.db** obter informação sobre as condições meteorológicas apresentadas.

From PIL import Image

```
from PIL import Image
```

PIL (Python Imaging Library) é uma biblioteca que fornece umas inúmeras funcionalidades para trabalhar com imagens. A classe vai permitir realizar várias operações como, por exemplo, inserir e redimensionar.

From time import sleep

```
from time import sleep
```

O modulo ***time*** é uma biblioteca que fornece várias funções relacionadas ao tempo, como medir o tempo de execução, como é usado neste caso.

A função ***sleep*** aceita um argumento que representa o número de segundos que o programa deve pausar.

From tkinter import messagebox

```
from tkinter import messagebox
```

O ***messagebox*** fornece uma série de funções que permitem exibir caixas de diálogo ***janelas pop-up*** para mostrar mensagens ao utilizador, como alertas, confirmações e erros. Essas caixas de diálogo são úteis para interagir com o utilizador e fornecer feedback sobre ações realizadas na aplicação.

From datetime import datetime

```
from datetime import datetime
```

A classe ***datetime*** combina informações de data e hora em um único objeto. Ela permite que criemos, manipulemos e formatemos datas e horas, além de poder realizar operações como a adição e subtração de tempo. Neste caso foi usado para saber os dias que vão aparecer quando pesquisar o tempo.

Base de dados

Classe BaseDados é o componente responsável pela gestão persistente dos dados da aplicação, utilizando uma base de dados SQLite local denominada Base_de_Dados.db. esta classe assegura a criação, manutenção e consulta das tabelas fundamentais para o funcionamento do sistema.

`class BaseDados:`

Estrutura da base de dados

A base de dados contém duas tabelas principais:

Utilizadores: armazena os dados dos utilizadores registados, incluindo um identificador único (ID), nome de utilizador, palavra-passe, UID, email e localização.

Table: Utilizadores

Fields

Name	Type	NN	PK	AI	U	Default	Check
ID	INTEGER	☒	☒	☒	☐	☐	☐
Nome_de_Utilizador	TEXT	☐	☐	☐	☒	☐	☐
Senha	TEXT	☒	☐	☐	☐	☐	☐
UID	INTEGER	☐	☐	☐	☒	☐	☐
Email	TEXT	☐	☐	☐	☒	☐	☐
Localizacao	TEXT	☐	☐	☐	☐	☐	☐

```

1 CREATE TABLE "Utilizadores" (
2   "ID" INTEGER,
3   "Nome_de_Utilizador" TEXT UNIQUE,
4   "Senha" TEXT NOT NULL,
5   "UID" INTEGER UNIQUE,
6   "Email" TEXT UNIQUE,
7   "Localizacao" TEXT,
8   PRIMARY KEY("ID")
9 );

```

Histórico: regista o histórico das pesquisas meteorológicas realizadas por cada utilizador, com data da pesquisa, localização, temperaturas mínimas e máximas, precipitação, descrição do tempo e o tipo de pesquisa que foi feita.

Table: Historico

Fields

Name	Type	NN	PK	AI	U	Default	Check
ID	INTEGER	☒	☒	☒	☐	☐	☐
Data	TEXT	☒	☐	☐	☐	☐	☐
UID	INTEGER	☐	☐	☐	☒	☐	☐
Localizacao	TEXT	☒	☐	☐	☐	☐	☐
Temperatura_Min	REAL	☐	☐	☐	☐	☐	☐
Temperatura_Max	REAL	☐	☐	☐	☐	☐	☐

```

1 CREATE TABLE "Historico" (
2   "ID" INTEGER,
3   "Data" TEXT NOT NULL,
4   "UID" INTEGER,
5   "Localizacao" TEXT NOT NULL,
6   "Temperatura_Min" REAL,
7   "Temperatura_Max" REAL,
8   "Precipitacao" REAL,
9   "Descricao" TEXT,
10  "Tipo_Procura" TEXT,
11  PRIMARY KEY("ID")

```

As tabelas são criadas automaticamente pelo método `criar_tabelas()`, garantindo que a base de dados está pronta para uso na inicialização da aplicação.

```
def criar_tabelas(self):
    conn = self.conectar_base_de_dados()
    cursor = conn.cursor()

    # Criação da tabela de utilizadores
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS Utilizadores (
            ID INTEGER PRIMARY KEY,
            Nome_de_Utilizador TEXT UNIQUE,
            Senha TEXT NOT NULL,
            UID INTEGER UNIQUE,
            Email TEXT UNIQUE,
            Localizacao TEXT
        )
    ''')

    # Criação da tabela de histórico associada a cada utilizador via UID
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS Historico (
            ID INTEGER PRIMARY KEY,
            Data TEXT NOT NULL,
            UID INTEGER,
            Localizacao TEXT NOT NULL,
            Temperatura_Min REAL,
            Temperatura_Max REAL,
            Precipitacao REAL,
            Descricao TEXT,
            Tipo_Procura TEXT,
            FOREIGN KEY(UID) REFERENCES Utilizadores(UID)
        )
    ''')

    conn.commit() # Confirma as alterações
    conn.close()  # Fecha a ligação
```

Funcionalidades principais

Inserção de utilizadores: o método `inserir_utilizador()` permite adicionar novos utilizadores, evitando haver duplicações ao verificar previamente a existência de registos com o mesmo nome, UID ou email.

```
def inserir_utilizador(self, utilizador):
    if not utilizador:
        return False # Verifica se o objeto utilizador é válido

    # Verifica se o utilizador já existe (por nome, UID ou e-mail)
    if self.verificar_existencia(utilizador, utilizador.mail):
        return False # Se já existe, não insere na base de dados

    conn = self.conectar_base_de_dados()
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO Utilizadores (Nome_de_Utilizador, Senha, UID, Email, Localizacao)
        VALUES (?, ?, ?, ?, ?)
    ''', (utilizador.nome, utilizador.senha, utilizador.uid, utilizador.mail, utilizador.localizacao))
    conn.commit()
    conn.close()
    return True
```

Autenticação: através do método login (), a aplicação valida as credenciais do utilizador, permitindo o acesso apenas caso o nome ou UID e a palavra-passe estejam corretos.

```
def login(self, utilizador):
    conn = self.conectar_base_de_dados()
    cursor = conn.cursor()

    cursor.execute('''
        SELECT * FROM Utilizadores WHERE (Nome_de_Utilizador = ? OR UID = ?) AND Senha = ?
    ''', (utilizador.nome, utilizador.nome, utilizador.senha))
    resultado = cursor.fetchone()

    # Se não encontrou por nome ou UID, tenta caso especial: UID == senha (ambos numéricos)
    if not resultado:
        try:
            uid = int(utilizador.nome)
            if str(uid) == utilizador.senha:
                cursor.execute('SELECT * FROM Utilizadores WHERE UID = ?', (uid,))
                resultado = cursor.fetchone()
        except ValueError:
            pass # Se não for número, ignora

    conn.close()
    return resultado is not None # True se login for bem-sucedido
```

Gestão de senhas: inclui funcionalidades para verificar a existência de um email (verificar-mail) e para alteração da palavra-passe associada a um utilizador (alterar senha).

```
def verificar_mail(self, email):
    conn = self.conectar_base_de_dados()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM Utilizadores WHERE Email = ?", (email,))
    resultado = cursor.fetchone()
    conn.close()
    return resultado is not None

def alterar_senha(self, email, nova_senha):
    conn = self.conectar_base_de_dados()
    cursor = conn.cursor()
    cursor.execute("UPDATE Utilizadores SET Senha = ? WHERE Email = ?", (nova_senha, email))
    conn.commit()
    conn.close()
```

Histórico de pesquisa: A função `guardar_historico ()` regista as pesquisas efetuadas, armazenando os detalhes meteorológicos para posterior consulta, a qual é facilitada por outra função, a `obter_ultimos_registos ()`, que nos vai retornar os dados das últimas pesquisas que foram feitas nos últimos 30 dias.

```
def guardar_historico(self, uid, historico):
    data_atual = datetime.now().strftime("%Y-%m-%d %H:%M:%S") # Data e hora atual
    conn = self.conectar_base_de_dados()
    cursor = conn.cursor()

    cursor.execute('''
        INSERT INTO Historico (Data, UID, Localizacao, Temperatura_Min, Temperatura_Max, Precipitacao, Descricao, Tipo_Procura)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    ''', (
        data_atual, uid, historico.localizacao, historico.temperatura_min,
        historico.temperatura_max, historico.precipitacao,
        historico.descricao, historico.tipo_procura
    ))

    conn.commit()
    conn.close()
```

```
def obter_ultimos_registos(self, uid, tipo_procura):
    conn = self.conectar_base_de_dados()
    cursor = conn.cursor()
    try:
        cursor.execute('''
            SELECT Temperatura_Min, Temperatura_Max, Precipitacao
            FROM Historico
            WHERE UID = ? AND Tipo_Procura = ? AND date(Data) >= date('now', '-30 days')
            ORDER BY Data DESC
        ''', (uid, tipo_procura))

        return cursor.fetchall() # Retorna lista de tuplas com os dados
    except Exception:
        return []
    finally:
        conn.close()
```

API

Módulo de integração com a API do IPMA

A classe, classe_ipma, foi criada com o propósito de comunicar diretamente com a API pública que foi disponibilizada pelo mesmo site do IPMA, que vai permitir que possamos obter informação meteorológica atualizada e estruturada, que é essencial para o funcionamento da nossa aplicação.

```
class classe_ipma:
```

Esta classe contém apenas métodos estáticos, significa que não é preciso instanciar os objetos para poder aceder às suas funcionalidades. O design vai simplificar a consulta de dados externos e a melhoria da organização do código.

Funcionalidades principais

Obter todas as nossas localidades

Com a função obter_todas_localidades vai realizar uma chamada à API (<https://api.ipma.pt/public-data/forecast/locations.json>), que nos devolve a lista de todas as localidades para as quais estão disponíveis as previsões meteorológicas.

```
def obter_todas_localidades():  
    url = 'https://api.ipma.pt/public-data/forecast/locations.json' # URL da API para obter localidades  
    try:  
        resposta = requests.get(url) # Faz a requisição GET  
        if resposta.status_code == 200: # Verifica se a resposta foi bem-sucedida (HTTP 200)  
            return resposta.json() # Retorna os dados em formato JSON  
    except:  
        pass # Em caso de erro (ex: sem internet), ignora a exceção  
    return None # Retorna None se não obtiver sucesso
```

Obter a previsão agregada

Na função obter_previsao_aggregate (global_id_local) utilizamos o global_id_local das cidades e localidades para conseguirmos obter as previsões meteorológicas agregadas.

```
def obter_previsao_aggregate(global_id_local):  
    # URL para obter previsao metereologica de acordo com o id  
    url = f'https://api.ipma.pt/public-data/forecast/aggregate/{global_id_local}.json'  
    try:  
        resposta = requests.get(url)  
        if resposta.status_code == 200:  
            return resposta.json()  
    except:  
        pass  
    return None
```

Obter ID Global de uma Localidade

Na função obter_global_id_local (cidade) foi feita para fazer a pesquisa, na lista de localidades apresentadas, o identificador (globalIdLocal) correspondente a uma cidade fornecida pelo utilizador. Este ID é essencial para conseguirmos obter as previsões de forma precisa.

```
def obter_global_id_local(cidade):  
    localidades = classe_ipma.obter_todas_localidades() # Obtém todas as localidades  
    if localidades is None:  
        return None # Se não conseguir obter localidades, retorna None  
    for loc in localidades:  
        # Verifica se o nome da localidade corresponde à cidade procurada (ignorando maiúsculas/minúsculas e espaços)  
        if loc['local'] and loc['local'].strip().lower() == cidade.strip().lower():  
            return loc['globalIdLocal'] # Retorna o ID correspondente  
    return None # Se não encontrar a cidade, retorna None
```

Dicionário e Funções Auxiliares

O dicionário distritos_por_id é auxiliar para associar os identificadores numéricos de distritos (idDistrito) aos seus nomes, incluindo distritos dos arquipélagos, a Madeira e os Açores.

```
distritos_por_id = {  
    1: "Aveiro", 2: "Beja", 3: "Braga", 4: "Bragança", 5: "Castelo Branco",  
    6: "Coimbra", 7: "Évora", 8: "Faro", 9: "Guarda", 10: "Leiria",  
    11: "Lisboa", 12: "Portalegre", 13: "Porto", 14: "Santarém",  
    15: "Setúbal", 16: "Viana do Castelo", 17: "Vila Real", 18: "Viseu",  
    31: "Madeira", 41: "Açores - Vila do Porto", 42: "Açores - Nordeste / Ponta Delgada",  
    43: "Açores - Angra do Heroísmo", 44: "Açores - Santa Cruz da Graciosa",  
    45: "Açores - Velas (São Jorge)", 46: "Açores - Madalena (Pico)",  
    47: "Açores - Horta (Faial)", 48: "Açores - Santa Cruz das Flores",  
    49: "Açores - Vila do Corvo"  
}
```

Função extrair_distritos (localidades)

Extrai todos os identificadores de distrito únicos de uma lista de localidades, garantindo que não vai haver repetições e devolve-nos ordenados.

```
def extrair_distritos(localidades):  
    distritos = set() # Usa um conjunto para garantir unicidade  
    for loc in localidades:  
        distritos.add(loc['idDistrito']) # Adiciona o ID do distrito à coleção  
    return sorted(distritos) # Retorna a lista ordenada de IDs
```

Função filtrar_localidades_por_distrito (localidades, distrito_id)

Filtra e devolve apenas as localidades que pertencem a um determinado distrito.

```
def filtrar_localidades_por_distrito(localidades, distrito_id):  
    return [loc for loc in localidades if loc['idDistrito'] == distrito_id] # Lista filtrada
```

Caixa de texto

A classe `CaixaInformacoes` é responsável por apresentar, no ecrã principal da aplicação, uma caixa de texto informativo com o formato visual agradável e funcional. Esta caixa é criada dentro de um frame específico e utiliza um componente gráfico do tipo `CTkTextbox`, com definições personalizadas de tamanho, cor, margem e tipo de letra.

```
class CaixaInformacoes:
```

```
    def __init__(self, frame):
        self.frame = frame # Guarda o frame onde a caixa de texto será colocada
        self.textbox = None # Inicializa a variável da caixa de texto como None
        self.criar_caixa_texto() # Chama o método para criar e mostrar a caixa de texto

    def criar_caixa_texto(self):
        # Cria uma CTkTextbox dentro do frame fornecido
        self.textbox = ctk.CTkTextbox(
            self.frame,
            width=300, height=350,
            corner_radius=10,
            border_width=2,
            border_color="white",
            wrap="word",
            font=("Segoe UI", 12)
        )
        self.textbox.place(relx=0.5, rely=0.5, anchor="center")

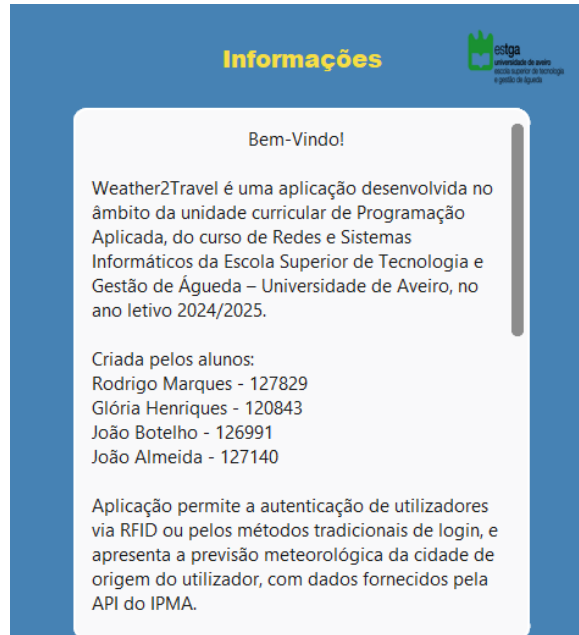
        self.inserir_texto()
```

O conteúdo da caixa inclui uma mensagem de boas-vindas, seguida de uma

```
# Insere um cabeçalho de boas-vindas
self.textbox.insert("end", "                                Bem-Vindo!\n\n")

# Insere a descrição da aplicação e os autores
self.textbox.insert("end",
    "Weather2Travel é uma aplicação desenvolvida no âmbito da unidade curricular de "
    "Programação Aplicada, do curso de Redes e Sistemas Informáticos da Escola Superior de "
    "Tecnologia e Gestão de Águas da Universidade de "
    "Aveiro, no ano letivo 2024/2025.\n\nCriada pelos alunos:\nRodrigo Marques - 127829\nGlória Henriques - 120843\n"
    "João Botelho - 126991\nJoão Almeida - 127140\n\nA aplicação permite a autenticação de utilizadores via RFID "
    "ou pelos métodos tradicionais de login, e "
    "apresenta a previsão meteorológica da cidade de origem do utilizador, com dados fornecidos "
    "pela API do IPMA.\n\nEste projeto representa a consolidação prática de vários conceitos "
    "fundamentais da programação orientada a objetos, integração de sistemas, gestão de base de "
    "dados e desenvolvimento de interfaces gráficas, refletindo o esforço colaborativo e o "
    "compromisso académico dos seus autores.\n\n"
)
```

descrição detalhada do propósito da aplicação, a unidade curricular, a nossa identificação e explicação sobre as funcionalidades principais.



Adicionalmente, são incluídas notas importantes relativas à criação de conta, com especial destaque para os requisitos de segurança da senha e instruções sobre o uso do UID como método alternativo de autenticação.

<pre>self.textbox.insert("end", "=====\n" " Notas a Reter\n" "=====\n" "Ao criar conta, tem em Atenção os cuidados a ter em alguns tópicos.\n\n" "Senha - A sua senha deve conter:\n" "-> Pelo menos 8 caracteres;\n" "-> Pelo menos uma letra maiúscula;\n" "-> Pelo menos uma letra minúscula;\n" "-> Pelo menos um número;\n\n" "Caso queira Iniciar Sessão apenas com UID, ao criar conta preencha os campos UID e Senha " "com o UID. Após criar a conta será possível Iniciar Sessão apenas com UID no campo " "Nome de Utilizador / UID e clicando no botão Login.\nComo deve saber, esta forma não é " "segura, logo fica ao seu critério.\n\nVersão em " "vigor: 1.0")</pre>	<pre>----- Notas a Reter ===== Ao criar conta, tem em Atenção os cuidados a ter em alguns tópicos. Senha - A sua senha deve conter: -> Pelo menos 8 caracteres; -> Pelo menos uma letra maiúscula; -> Pelo menos uma letra minúscula; -> Pelo menos um número; Caso queira Iniciar Sessão apenas com UID, ao criar conta preencha os campos UID e Senha com o UID. Após criar a conta será possível Iniciar Sessão apenas com UID no campo Nome de Utilizador / UID e clicando no botão Login. Como deve saber, esta forma não é segura, logo fica ao seu critério. Versão em vigor: 1.0</pre>
---	---

No final a caixa é configurada como “somente leitura”, impedindo que o utilizador edite o conteúdo. Esta funcionalidade serve como uma introdução clara e útil para qualquer utilizador que abra a aplicação pela sua primeira vez.

Torna a caixa de texto não editável ou seja somente leitura

```
self.textbox.configure(state="disabled")
```

Módulo auxiliar

Classe histórico

A classe representa um registo individual de uma pesquisa meteorológica efetuada pelo utilizador. Cada instância desta classe guarda os dados principais de uma previsão, incluindo o nome da localidade, a temperatura mínima e máxima prevista, o valor da precipitação, uma descrição textual do estado do tempo, e ainda o tipo de procura efetuada (por exemplo, se foi manual ou automática). Esta estrutura vai facilitar o armazenamento e a consulta posterior do histórico de pesquisas na aplicação.

```
class Historico:
    def __init__(self, localizacao, temperatura_min, temperatura_max, precip, descricao, tipo_procura):
        # Inicializa os atributos de um registo histórico meteorológico
        self.localizacao = localizacao
        self.temperatura_min = temperatura_min
        self.temperatura_max = temperatura_max
        self.precipitacao = precip
        self.descricao = descricao
        self.tipo_procura = tipo_procura
```

Classe utilizador

Define a estrutura base de um utilizador da aplicação. Cada objeto desta classe contém os dados essenciais associados ao perfil de utilizador, a senha de acesso, o identificador RFID (UID), o endereço email e a localidade de origem definida. Estes atributos permitem não só a autenticação e recuperação de conta, como também a personalização da previsão meteorológica com base na localização do utilizador.

```
class Utilizador:
    def __init__(self, nome, senha, uid, mail, localizacao):
        # Inicializa os atributos de um utilizador
        self.nome = nome
        self.senha = senha
        self.uid = uid
        self.mail = mail
        self.localizacao = localizacao
```

Classe obter_imagens

Tem como principal função gerir o carregamento e a exibição de imagens dentro da interface gráfica da aplicação. Ao ser inicializada, determina automaticamente o diretório, base de onde os recursos devem ser lidos, garantindo compatibilidade tanto em modo de desenvolvimento como em versões empacotadas (ex. com PyInstaller).

```
class Obter_imagens:
    def __init__(self):
        # Define a base de onde os recursos (imagens) serão carregados
        if hasattr(sys, '_MEIPASS'):
            self.base = sys._MEIPASS # Para aplicações empacotadas como com o PyInstaller
        else:
            self.base = os.path.abspath(".")

    def caminho_recurso(self, caminho_relativo):
        # Retorna o caminho absoluto de um ficheiro a partir do caminho relativo
        return os.path.join(self.base, caminho_relativo)

    def config_imagem(self, png, frame, size, relx, rely, bg_color=None):
        # Carrega uma imagem e coloca-a no frame especificado
        try:
            caminho_imagem = self.caminho_recurso(png)
            imagem = ctk.CTkImage(Image.open(caminho_imagem), size=size)
            label_logo = ctk.CTkLabel(frame, image=imagem, text="", bg_color=bg_color)
            label_logo.place(relx=relx, rely=rely, anchor="center")
        except Exception as e:
            # Mostra um erro se não for possível carregar a imagem
            messagebox.showerror(f"Erro ao carregar imagem", f"Erro ao carregar imagem '{png}': {e}")
```

Dispõe de dois métodos principais:

Caminho recurso (caminho relativo): devolve o caminho absoluto de uma imagem ou recurso com base num caminho relativo.

```
def caminho_recurso(self, caminho_relativo):
    # Retorna o caminho absoluto de um ficheiro a partir do caminho relativo
    return os.path.join(self.base, caminho_relativo)
```

Config imagem (...): carrega e insere uma imagem num determinado frame da interface gráfica, com dimensões e posicionamento personalizáveis.

```
def config_imagem(self, png, frame, size, relx, rely, bg_color=None):
    # Carrega uma imagem e coloca-a no frame especificado
    try:
        caminho_imagem = self.caminho_recurso(png)
        imagem = ctk.CTkImage(Image.open(caminho_imagem), size=size)
        label_logo = ctk.CTkLabel(frame, image=imagem, text="", bg_color=bg_color)
        label_logo.place(relx=relx, rely=rely, anchor="center")
    except Exception as e:
        # Mostra um erro se não for possível carregar a imagem
        messagebox.showerror(f"Erro ao carregar imagem", f"Erro ao carregar imagem '{png}': {e}")
```

Vai nos permitir manter a organização e reutilização eficiente de imagens ao longo de toda a aplicação.

Classe remover_frames

Fornece-nos uma funcionalidade auxiliar simples, mas essencial: a remoção de todos os widgets (elementos gráficos) de um determinado frame da interface gráfica.

```
class Remover_frames:  
    @staticmethod  
    def limpar_widgets_do_frame(frame):  
        # Remove todos os widgets de um frame  
        for widget in frame.winfo_children():  
            widget.destroy()
```

É composta por um único método estático:

Limpar widgets do frame (frame): percorre todos os widgets contidos no frame indicado e remove-os permanentemente, utilizando o método destroy () de cada widget.

```
def limpar_widgets_do_frame(frame):  
    # Remove todos os widgets de um frame  
    for widget in frame.winfo_children():  
        widget.destroy()
```

Esta funcionalidade é útil sempre que pretendemos “limpar” um ecrã antes de apresentar novos conteúdos ou reconfigurar a interface, evitando a sobreposição de elementos.

Classe Esconder_frames

Esta classe tem um propósito semelhante à anterior, mas com uma diferença, importante: em vez de remover os widgets de um frame, esta classe apenas os esconde temporariamente.

```
class Esconder_frames:  
    @staticmethod  
    def limpar_widgets_do_frame(frame):  
        # Esconde todos os widgets de um frame  
        for widget in frame.winfo_children():  
            widget.forget()
```

Tem só uma função estática:

Limpar widgets do frame (frame): percorre todos os widgets contidos no frame fornecido e esconde-os da interface, utilizando o método `forget()` de cada widget. Isso retira os elementos da visualização sem os destruir, o que permite voltar a mostrá-los posteriormente se necessário.

```
def limpar_widgets_do_frame(frame):  
    # Esconde todos os widgets de um frame  
    for widget in frame.winfo_children():  
        widget.forget()
```

Esta abordagem é útil em situações em que se pretende alterar rapidamente entre diferentes conteúdos da interface, preservando o estado interno dos widgets.

Classe visualizar_senha

A classe foi criada para permitir alternar entre mostrar e ocultar o conteúdo de um campo de senha numa interface gráfica, oferecendo uma funcionalidade comum em aplicações modernas.

```
class Visual_senha:  
    def __init__(self, entrada, botao):  
        self.entrada = entrada # Campo de entrada de texto (password)  
        self.botao = botao     # Botão para alternar visibilidade da password  
        self.estado = False    # Estado inicial: senha oculta  
  
    def alternar_modos_visual_senha(self):  
        if self.estado:  
            # Tornar senha oculta  
            self.entrada.configure(show="*")  
            self.botao.configure(text="👁")  
            self.estado = False  
        else:  
            # Tornar senha visível  
            self.entrada.configure(show="")  
            self.botao.configure(text="🔒")  
            self.estado = True
```

A classe irá funcionar da seguinte forma:

Possui um atributo estado, que controla se a senha está visível (True) ou oculta (False).

O método `alternar_modo_visual_senha()` altera este estado:

```
def alternar_modo_visual_senha(self):  
    if self.estado:  
        # Tornar senha oculta  
        self.entrada.configure(show="*")  
        self.botao.configure(text="👁")  
        self.estado = False  
    else:  
        # Tornar senha visível  
        self.entrada.configure(show="")  
        self.botao.configure(text="👤")  
        self.estado = True
```

Se a senha estiver visível, volta a ocultá-la (`show="*"`), e muda o ícone do botão para "👁";

```
self.entrada.configure(show="*")  
self.botao.configure(text="👁")  
self.estado = False
```



Se estiver oculta, torna-a visível (`show=""`), e altera par o ícone "👤".

```
self.entrada.configure(show="")  
self.botao.configure(text="👤")  
self.estado = True
```



Isto permite ao utilizador verificar o que escreveu no campo de senha, melhorando a usabilidade da aplicação sem comprometer a segurança por defeito.

Classe *EmailConfirmacao*

Tem como objetivo principal gerar e enviar um código de confirmação para o email do utilizador. Esta classe está configurada para utilizar o servidor SMTP do Gmail com ligação segura (SSL).

```
class EmailConfirmacao:
    def __init__(self):
        # Configuração do servidor de envio de e-mails
        self.EMAIL_REMETENTE = "weather2travel.senhas@gmail.com"
        self.SENHA_APP = "ehbf gvdz uzbd pcuz" # Senha da app (não a senha da conta!)
        self.SERVIDOR_SMTP = "smtp.gmail.com"
        self.PORTA_SMTP = 465

    def gerar_codigo(self):
        # Gera um código de 4 dígitos para verificação
        return str(random.randint(1000, 9999))

    def enviar(self, email_destino):
        # Envia o código para o e-mail fornecido
        codigo = self.gerar_codigo()
        assunto = "Código de confirmação de conta - Weather2Travel"
        mensagem = (f"Código necessário para confirmar a sua conta na App Weather2Travel:\n\n{codigo}")

        try:
            msg = EmailMessage()
            msg["From"] = self.EMAIL_REMETENTE
            msg["To"] = email_destino
            msg["Subject"] = assunto
            msg.set_content(mensagem)

            with smtplib.SMTP_SSL(self.SERVIDOR_SMTP, self.PORTA_SMTP) as smtp:
                smtp.login(self.EMAIL_REMETENTE, self.SENHA_APP)
                smtp.send_message(msg)

            return codigo # Retorna o código para validação posterior
        except Exception:
            return # Em caso de erro, retorna None
```

A função `gerar_codigo ()` cria um código numérico aleatório de quatro dígitos, que será enviado para o email do utilizador.

```
def gerar_codigo(self):
    # Gera um código de 4 dígitos para verificação
    return str(random.randint(1000, 9999))
```

O método `enviar (email_destino)` constrói a mensagem do email, realiza a autenticação no servidor SMTP usando as credenciais da aplicação e envia o email para o destinatário. Caso o envio seja bem-sucedido, este método retorna

Caso o envio seja bem-sucedido, este método retorna o código gerado para que seja usado na validação posterior. Se ocorrer algum erro durante o envio, o método retorna `None`.

Caso o envio seja bem-sucedido, este método retorna o código gerado para que ser usado na validação posterior. Se ocorrer algum erro durante o envio, o método retorna None.

Classe *EnviarEmailVerificacao*

A classe é responsável pelo envio de um código de verificação para os utilizadores que pretendem alterar a sua palavra-passe.

```
class EnviarEmailVerificacao:
    def __init__(self, frame_fundo, entrada_email, label_avisos, bd, func_mostrar_frame_codigo):
        self.frame_fundo = frame_fundo
        self.entrada_email = entrada_email
        self.label_avisos = label_avisos
        self.bd = bd
        self.func_mostrar_frame_codigo = func_mostrar_frame_codigo

        self.email_remetente = "weather2travel.senhas@gmail.com"
        self.senha_app = "ehbf gvdz uzbd pcuz"

    def enviar_mail(self):
        numero_mail = str(random.randint(1000, 9999))
        assunto = "Código de confirmação de conta - Weather2Travel"
        mensagem = f"Código necessário para mudar a senha da sua conta:\n\n{numero_mail}"

        destinatario = self.entrada_email.get().strip()

        if not destinatario:
            self.label_avisos.configure(text="*Este campo é Obrigatório!", text_color="red")
            return

        if not self.bd.verificar_mail(destinatario):
            messagebox.showerror("E-mail Inválido", "O E-mail não está associado a uma conta")
            return

        try:
            msg = EmailMessage()
            msg["From"] = self.email_remetente
            msg["To"] = destinatario
            msg["Subject"] = assunto
            msg.set_content(mensagem)

            with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
                smtp.login(self.email_remetente, self.senha_app)
                smtp.send_message(msg)

            self.label_avisos.configure(text="A enviar e-mail...", text_color="white")
            self.frame_fundo.update()
            sleep(1)

            self.label_avisos.configure(text="Mail enviado com sucesso!\nVerifique o seu e-mail.", text_color="green")
            self.frame_fundo.update()
            sleep(2)

            self.func_mostrar_frame_codigo(numero_mail)

        except Exception:
            self.label_avisos.configure(text="Insira um mail válido!", text_color="red")
```

Esta classe recebe como parâmetros vários elementos da interface gráfica, a base de dados e uma função para mostrar o frame onde o código deve ser inserido.

Na função `enviar_mail()`, é gerado um código de quatro dígitos e verificados se o email introduzido está preenchido e está associado a uma conta registada na base de dados.

```
def enviar_mail(self):
    numero_mail = str(random.randint(1000, 9999))
    assunto = "Código de confirmação de conta - Weather2Travel"
    mensagem = f"Código necessário para mudar a senha da sua conta:\n\n{numero_mail}"

    destinatario = self.entrada_email.get().strip()

    if not destinatario:
        self.label_aviso.configure(text="*Este campo é Obrigatório!", text_color="red")
        return

    if not self.bd.verificar_mail(destinatario):
        messagebox.showerror("E-mail Inválido", "O E-mail não está associado a uma conta")
        return

    try:
        msg = EmailMessage()
        msg["From"] = self.email_remetente
        msg["To"] = destinatario
        msg["Subject"] = assunto
        msg.set_content(mensagem)

        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
            smtp.login(self.email_remetente, self.senha_app)
            smtp.send_message(msg)

        self.label_aviso.configure(text="A enviar e-mail...", text_color="white")
        self.frame_fundo.update()
        sleep(1)

        self.label_aviso.configure(text="Mail enviado com sucesso!\nVerifique o seu e-mail.", text_color="green")
        self.frame_fundo.update()
        sleep(2)

        self.func_mostrar_frame_codigo(numero_mail)

    except Exception:
        self.label_aviso.configure(text="Insira um mail válido!", text_color="red")
```

Caso o email seja válido, é enviado um email com o código de verificação e a interface gráfica é atualizada para informar o utilizador sobre o estado do envio. Em caso de erro, como um email inválido ou vazio, são apresentados avisos apropriados ao utilizador para corrigir esta situação.

Esta classe também utiliza o servidor SMTP do Gmail para o envio dos emails de forma mais segura, garantido uma comunicação protegida.

Interface gráfica

Classe App

É a principal de todas e é responsável por inicializar e gerir a interface gráfica da aplicação Weather2Travel, utilizando a biblioteca CustomTkinter.

Função `__init__`

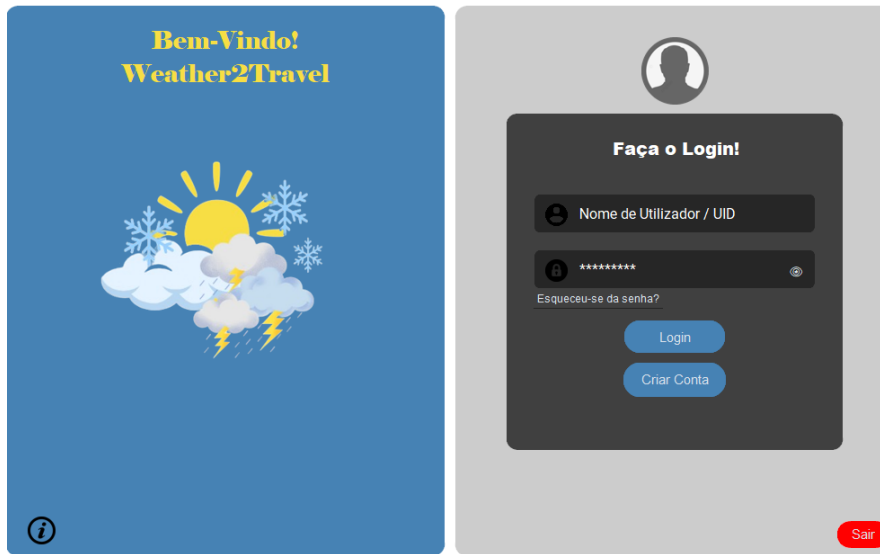
```
def __init__(self):
    self.dados = model.Obter_imagens()      # Objeto responsável por fornecer imagens e caminhos
    self.janela = ctk.CTk()                 # Janela inicial da aplicação
    self.janela.title("Weather2Travel")      # Título da janela
    self.janela.geometry("800x500")         # Dimensões da janela
    self.janela.resizable(False, False)     # Desabilita o redimensionamento da janela
    self.janela.configure(fg_color="white")  # Cor de fundo da janela

    self.func_frame_boasvindas()             # Chama a função para criar o frame de boas-vindas
    self.func_frame_login()                 # Chama a função para o frame de login
    self.janela.mainloop()                  # Inicia o loop principal da aplicação
```

No construtor da classe App, é criada a janela principal da aplicação, configurando o título, as dimensões fixas, a cor de fundo e desativando o redimensionamento. É criado um objeto para gerir imagens e são chamados os métodos para criar o frame de boas-vindas e o frame de login. Finalmente, inicia-se o loop principal da interface para manter a aplicação ativa.

Função `func_frame_boasvindas`

Esta função cria o frame de boas-vindas com fundo azul, bordas arredondadas e um contorno branco. Dentro do frame, adiciona um título de receção em fonte destacada e cor amarela, bem como uma imagem principal ilustrativa. Também cria um botão redondo com o ícone de informação, que ao ser clicado chama a função interna `func_frame_informacoes`.



Caso a imagem do botão não seja carregada corretamente, é mostrado um erro ao utilizador.

Função interna `func_frame_informacoes`

```
def func_frame_informacoes():
```

Esta função remove todos os widgets do frame de boas-vindas para limpar o conteúdo. Adiciona o botão “Voltar” que permite regressar ao frame inicial, exibindo novamente o conteúdo de boas-vindas. Exibe um título “Informações” destacado, apresenta o logotipo da ESTGA e mostra uma caixa de texto com informações úteis ao utilizador, utilizando a classe `CaixaInformacoes`.

`Model.Esconder_frames.limpar_widgets_do_frame (frame)`

Esta função externa é utilizada para eliminar todos os widgets presentes num determinado frame preparando o espaço para novos conteúdos sem sobreposição.

```
model.Remover_frames.limpar_widgets_do_frame(self.frame_boasvindas)
```

`Self.dados.caminho_recurso (caminho)`

Esta função devolve o caminho correto para um recurso, como uma imagem, facilitando o seu carregamento na aplicação mesmo em diferentes ambientes.

```
self.dados.config_imagem("imagens/logo_estga.png", self.frame_boasvindas, (65, 65), 0.85, 0.075, "steel blue")
```

Função `func_frame_login`

A função principal é responsável por apresentar o frame de login e, dentro dela, está definida uma função interna que cria o frame para registo de nova conta. Esta função interna começa por limpar todos os widgets existentes no frame login para preparar o ambiente para o formulário de criação de conta.

```
def func_frame_login(self):
```

Função `func_frame_criar_conta`

Em seguida, é criado um frame principal com estilo personalizado, onde são adicionados diversos elementos gráficos, incluindo etiquetas e caixas de entrada para o utilizador, senha, o UID, o email, bem como selecionar o distrito e a localidade através de menus dropdown. Para a senha é implementado um botão com um ícone de olho que permite alternar entre mostrar e esconder o texto da senha.

```
def func_frame_criar_conta():
```



Carregamento de dados da API e atualização dos menus dropdown

O código tenta carregar os dados das localidades e distritos a partir de uma API externa. Caso consiga obter esses dados, os menus dropdown são atualizados para refletir as opções disponíveis, permitindo que a escolha do distrito atualize automaticamente as localidades correspondentes. Se ocorrer algum problema ao carregar os dados, é mostrada uma mensagem de aviso ao utilizador.

Função `criar_conta`

Quando o utilizador clica para confirmar o email, é executada uma função que recolhe os valores preenchidos no formulário e realiza diversas validações.

```
def criar_conta():  
    # Dá às variáveis o valor inserido nas Entradas  
    nome = entrada_nome.get()  
    senha = entrada_senha.get()  
    uid = entrada_uid.get()  
    mail = entrada_mail.get()  
    localizacao = localidade_selecionada.get()
```

Verifica-se se o nome de utilizador não ultrapassa o limite de caracteres permitido, se todos os campos obrigatórios estão preenchidos, e se a senha cumpre requisitos mínimos de segurança, como conter letras maiúsculas, minúsculas e números, exceto quando a senha é igual ao UID, situação em que é aceite diretamente. Caso alguma destas condições não seja cumprida, uma mensagem de erro é exibida.

```
def validar_senha(senha):  
    # Permite explicitamente se a senha for igual ao UID  
    if senha == uid:  
        return True  
    if len(senha) < 8:  
        messagebox.showerror("Erro", "A senha deve ter pelo menos 8 caracteres!")  
        return False  
    if not any(char.isupper() for char in senha):  
        messagebox.showerror("Erro", "A senha deve ter pelo menos uma letra maiúscula!")  
        return False  
    if not any(char.islower() for char in senha):  
        messagebox.showerror("Erro", "A senha deve ter pelo menos uma letra minúscula!")  
        return False  
    if not any(char.isdigit() for char in senha):  
        messagebox.showerror("Erro", "A senha deve ter pelo menos um número!")  
        return False  
    return True
```



A senha deve ter pelo menos 8 caracteres!



A senha deve ter pelo menos uma letra minúscula!



A senha deve ter pelo menos uma letra maiúscula!



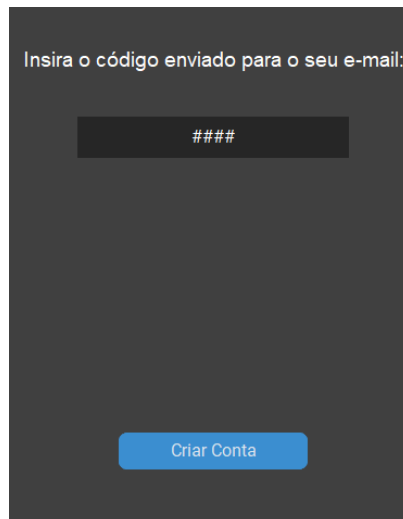
A senha deve ter pelo menos um número!

Se os dados forem válidos e o utilizador não existir já na base de dados, é enviado um código de confirmação para o email fornecido. Se o envio falhar, é exibida uma mensagem de erro.

Função mostrar_frame_codigo_confirmacao

Esta função apresenta um novo frame onde o utilizador deve inserir o código recebido por email para validar o registo.

```
def mostrar_frame_codigo_confirmacao():  
    frame_fundo = ctk.CTkFrame(self.frame_login, height=385, width=300, fg_color="gray25",  
                                corner_radius=0)  
    frame_fundo.place(relx=0.5, rely=0.525, anchor="center")  
  
    label_texto = ctk.CTkLabel(frame_fundo, text="Insira o código enviado para o seu e-mail:",  
                                font=("Arial", 15), text_color="white")  
    label_texto.place(relx=0.5, rely=0.1, anchor="center")  
  
    entrada_codigo = ctk.CTkEntry(frame_fundo, height=30, width=200,  
                                   placeholder_text="####",  
                                   text_color="white", placeholder_text_color="white", fg_color="gray15",  
                                   bg_color="gray15", border_width=0, justify="center")  
    entrada_codigo.place(relx=0.5, rely=0.25, anchor="center")  
  
    # Número de tentativas para inserir o código de confirmação  
    tentativas_restantes = [3]
```



O utilizador tem até três tentativas para acertar o código.

Se o código inserido for correto, o utilizador é registado na base de dados, recebe uma mensagem de sucesso e volta ao frame de login.

Se esgotar as tentativas sem sucesso, o processo é reiniciado.

Função verificar_codigo

Valida o código inserido pelo utilizador com o gerado previamente. Se for correto, cria a conta, insere-a na base de dados e redireciona para o login.

```
def verificar_codigo():
    codigo_inserido = entrada_codigo.get()
    if codigo_inserido == self.codigo_confirmacao:
        # Cria o utilizador após confirmação correta
        utilizador = model.Utilizador(nome, senha, uid, mail, localizacao)
        # Se os dados do utilizador foi adicionado à base de dados
        bd.inserir_utilizador(utilizador)
        messagebox.showinfo("Sucesso", "Conta criada com sucesso! Bem-vindo "
                                "à Weather2Travel!")
        # Voltar para o frame login
        self.func_frame_login()
    else:
        # Subtrai uma tentativa caso o código inserido não seja igual ao código gerado
        tentativas_restantes[0] -= 1
        # Caso o número de tentativas seja igual a 0 apresenta uma mensagem de erro
        if tentativas_restantes[0] == 0:
            messagebox.showerror("Erro",
                                "Número máximo de tentativas excedido. Reinicie o processo.")
            self.func_frame_login()
        # Se o número de tentativas ainda é maior que 0
        else:
            messagebox.showerror("Erro",
                                f"Código inválido. Tentativas restantes: {tentativas_restantes[0]}")
```



Código inválido. Tentativas restantes: 2



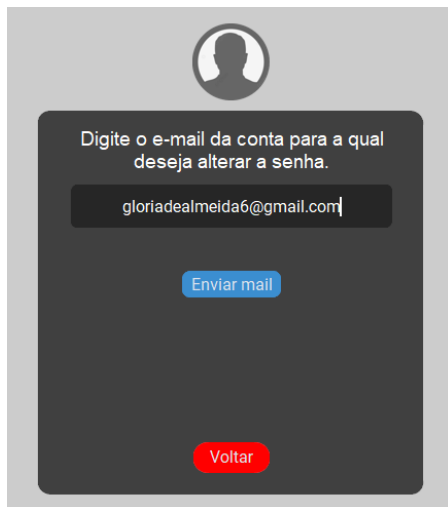
Conta criada com sucesso! Bem-vindo à Weather2Travel!

Caso contrário, decrementa o número de tentativas e exibe mensagens de erro apropriadas até ao limite de três tentativas.

Função `func_frame_mail_mudar_senha`

```
def func_frame_mail_mudar_senha():  
    model.Esconder_frames.limpar_widgets_do_frame(self.frame_login)  
  
    frame_fundo = ctk.CTkFrame(self.frame_login, height=300, width=300, fg_color="gray25", corner_radius=10)  
    frame_fundo.place(relx=0.5, rely=0.5, anchor="center")  
  
    self.dados.config_imagem("imagens/imagem_cadeado_passe.png", frame_fundo, (20, 20), 0.145, 0.25, "gray15")  
  
    label_texto = ctk.CTkLabel(frame_fundo,  
                               text="Digite o e-mail da conta para a qual\ndeseja alterar a senha.",  
                               font=("Arial", 15), text_color="white")  
    label_texto.place(relx=0.5, rely=0.1, anchor="center")  
  
    frame_mail = ctk.CTkFrame(frame_fundo, height=35, width=250, fg_color="gray15", corner_radius=5,  
                              bg_color="gray25")  
    frame_mail.place(relx=0.5, rely=0.25, anchor="center")  
  
    entrada_mail = ctk.CTkEntry(frame_fundo, height=30, width=200, placeholder_text="omeumail@gmail.com",  
                                text_color="white", placeholder_text_color="white", fg_color="gray15",  
                                bg_color="gray15", border_width=0, justify="center")  
    entrada_mail.place(relx=0.5, rely=0.25, anchor="center")  
  
    label_aviso = ctk.CTkLabel(frame_fundo, text="", font=("Arial", 10))  
    label_aviso.place(relx=0.5, rely=0.35, anchor="center")
```

Apresenta um formulário para o utilizador introduzir o e-mail da conta cuja senha pretende recuperar. Ao submeter, é enviado um código de verificação para o email indicado e é chamada a função `mostrar_frame_codigo`.



The screenshot shows a dark-themed window with a user icon at the top. Below it, a label asks the user to enter the email of the account they want to reset. An input field contains the email 'gloriadealmeida6@gmail.com'. There are two buttons: a blue 'Enviar mail' button and a red 'Voltar' button at the bottom.

Mail enviado com sucesso!
Verifique o seu e-mail



weather2travel.senhas@gmail.com

para mim ▼

Código necessário para mudar a senha da sua conta:

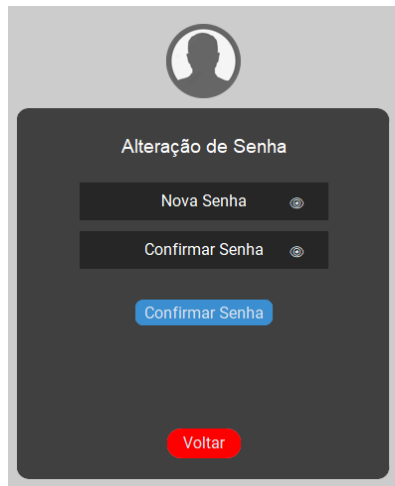
4575

Função mostrar_frame_codigo

```
def mostrar_frame_codigo(numero_mail, email_utilizador):  
    model.Esconder_frames.limpar_widgets_do_frame(self.frame_login)  
  
    frame_fundo = ctk.CTkFrame(self.frame_login, height=300, width=300, fg_color="gray25", corner_radius=10)  
    frame_fundo.place(relx=0.5, rely=0.5, anchor="center")  
  
    label_texto = ctk.CTkLabel(frame_fundo, text="Insira o código que foi enviado\npara o seu mail.",  
                               font=("Arial", 15), text_color="white")  
    label_texto.place(relx=0.5, rely=0.1, anchor="center")  
  
    frame_codigo = ctk.CTkFrame(frame_fundo, height=35, width=250, fg_color="gray15", corner_radius=5,  
                                bg_color="gray25")  
    frame_codigo.place(relx=0.5, rely=0.25, anchor="center")  
  
    entrada_codigo = ctk.CTkEntry(frame_fundo, height=30, width=200, placeholder_text="####",  
                                  text_color="white", placeholder_text_color="white",  
                                  fg_color="gray15", bg_color="gray15", border_width=0, justify="center")  
    entrada_codigo.place(relx=0.5, rely=0.25, anchor="center")  
  
    label_aviso_codigo = ctk.CTkLabel(frame_fundo, text="", font=("Arial", 10))  
    label_aviso_codigo.place(relx=0.5, rely=0.35, anchor="center")
```

Mostra o campo para inserção de código enviado por email.

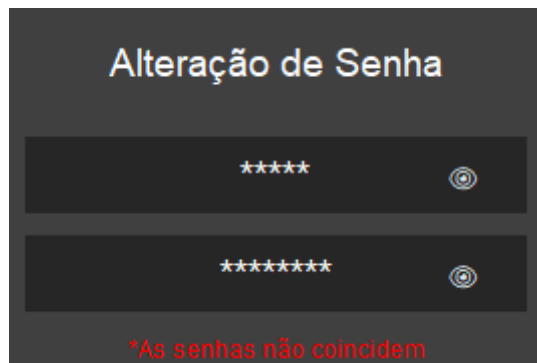
Se o código estiver correto, apresenta o formulário para introdução de nova senha.



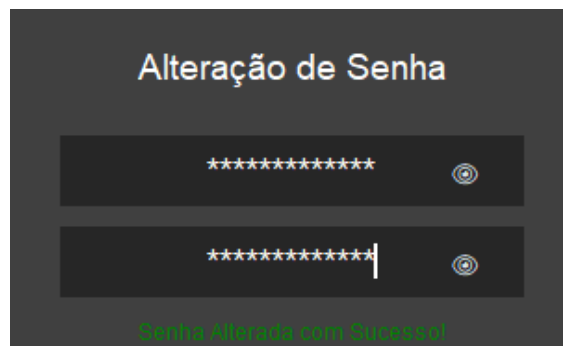
Caso contrário, permite até três tentativas antes de reiniciar o processo.

Função confirmar_senha

Valida e compara as novas senhas introduzidas.



Garante que a nova senha cumpre critérios de segurança (mínimo de 8 caracteres, maiúsculas, minúsculas e números).



Se tudo estiver correto, a senha é atualizada na base de dados e redireciona para o login.

Função verificar_login

Verifica se o nome e a senha inseridos estão corretos.

Se a senha estiver vazia. Tenta usar o nome como senha (login com UID).



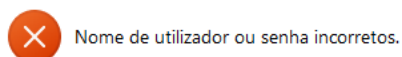
The image shows a login interface with a dark background. At the top, it says "Faça o Login!". Below this, there are two input fields. The first field has a user icon and contains the text "120843000". The second field has a lock icon and is empty, with a small eye icon to its right. Below the second field, there is a link that says "Esqueceu-se da senha?". At the bottom, there are two blue buttons: "Login" and "Criar Conta".

Caso contrário, faz login normal com o nome /UID e senha. Se os dados forem válidos, entra na aplicação com func_frame_APP (dados_utilizador).



The image shows the same login interface as before. The first input field now contains the text "gloria". The second input field has a lock icon and contains ten asterisks "*****", with a small eye icon to its right. The link "Esqueceu-se da senha?" and the buttons "Login" and "Criar Conta" remain the same.

Se não forem válidos, mostra uma mensagem de erro.



Função destruir_janela

Serve para terminar a aplicação, fechando a janela principal do login

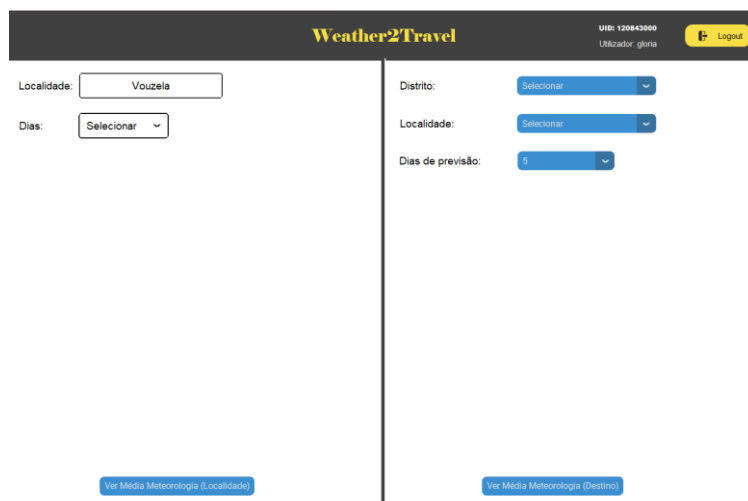
```
def destruir_janela():  
    self.janela.destroy()  
  
botao_sair = ctk.CTkButton(self.frame_login, text="Sair", font=("Arial", 12), fg_color="red", width=30,  
                           height=25,  
                           corner_radius=25, command=destruir_janela)  
botao_sair.place(relx=0.92, rely=0.95, anchor="center")
```

Função func_frame_APP

Cria a interface gráfica principal da aplicação após o login bem-sucedido.

```
def func_frame_APP(dados_utilizador):  
    # Obter esses dados para utilizar nesta nova função  
    uid, nome, localizacao = dados_utilizador  
    self.janela.destroy()  
    janela_APP = ctk.CTk()  
    janela_APP.title('Weather2Travel')  
    janela_APP.geometry('1080x720')  
    janela_APP.resizable(False, False)  
    janela_APP.configure(fg_color="white")  
  
    frame_cabecalho = ctk.CTkFrame(janela_APP, height=65, width=1080, fg_color="gray25", corner_radius=0)  
    frame_cabecalho.place(relx=0.5, rely=0.05, relwidth=1, relheight=0.1, anchor="center")  
  
    frame_divisao = ctk.CTkFrame(janela_APP, width=6, height=655, fg_color="gray25", corner_radius=0, border_width=0)  
    frame_divisao.place(relx=0.5, rely=0.55, anchor="center")  
  
    label_titulo = ctk.CTkLabel(frame_cabecalho, text="Weather2Travel", font=("elephant", 25),  
                               text_color="#F7DE44")  
    label_titulo.place(relx=0.5, rely=0.5, anchor="center")  
  
    label_uid_perfil = ctk.CTkLabel(frame_cabecalho, text=f"UID: {uid}", font=("Arial Black", 10),  
                                   text_color="white")  
    label_uid_perfil.place(relx=0.825, rely=0.35, anchor="center")  
  
    label_nome_perfil = ctk.CTkLabel(frame_cabecalho, text=f"Utilizador: {nome}", font=("Arial", 12),  
                                    text_color="white")  
    label_nome_perfil.place(relx=0.825, rely=0.65, anchor="center")
```

Esta função recebe os dados do utilizador (UID, nome e localização) e apresenta um novo ecrã com cabeçalho, separadores e menus suspensos para utilizador selecionar o distrito, localidade e número de dias para previsão meteorológica.



Inclui ainda uma função interna chamada logout, que permite encerrar a sessão atual, apresentado uma mensagem de confirmação e reiniciando a aplicação a partir da tela de login.

```
def logout():
    messagebox.showinfo("Sessão terminada com Sucesso", "Terminaste sessão com sucesso.")
    janela_APP.destroy()

    App()

frame_combobox_localidade = ctk.CTkFrame(janela_APP, fg_color="transparent")
frame_combobox_localidade.place(relx=0.25, rely=0.625, anchor="center")

frame_combobox_destino = ctk.CTkFrame(janela_APP, width=380, height=300, fg_color="transparent")
frame_combobox_destino.place(relx=0.75, rely=0.625, anchor="center")

label_distrito = ctk.CTkLabel(janela_APP, text="Distrito:", font=("Arial", 15),
                               fg_color="white", text_color="black")
label_distrito.place(relx=0.545, rely=0.15, anchor="center")

valor_distrito = ctk.StringVar(value="Selecionar")

menu_distrito = ctk.CTkOptionMenu(janela_APP, values=["Selecionar"],
                                   variable=valor_distrito, width=200)
menu_distrito.place(relx=0.77, rely=0.15, anchor="center")

label_localidade = ctk.CTkLabel(janela_APP, text="Localidade:", font=("Arial", 15),
                                fg_color="white", text_color="black")
label_localidade.place(relx=0.558, rely=0.225, anchor="center")

valor_num_dias = ctk.StringVar(value="5")
menu_num_dias = ctk.CTkOptionMenu(janela_APP, values=["1", "3", "5"], variable=valor_num_dias)
menu_num_dias.place(relx=0.743, rely=0.30, anchor="center")
```

Função atualizar_previsao_destino

```
def atualizar_previsao_destino(event=None):  
    # Obter localidades  
    localidades = API.classe_ipma.obter_todas_localidades()  
    if not localidades:  
        messagebox.showerror("Erro", "Erro ao obter imagens")  
        return  
  
    distritos_ids = API.extrair_distritos(localidades)  
  
    distritos_nome = [API.distritos_por_id[id_d] for id_d in distritos_ids if id_d in API.distritos_por_id]  
  
    # Atualiza o menu com os distritos  
    menu_distrito.configure(values=["Selecionar"] + distritos_nome)
```

É responsável por atualizar os menus suspensos de interface, permitindo ao utilizador escolher um distrito, uma localidade e visualizar a previsão meteorológica para essa zona.

Inicialmente, a função obtém todas as localidades disponíveis a partir da API. Com base nessas localidades, extrai os distritos e atualiza o menu correspondente com os nomes dos mesmos.

Dentro dessa função existem duas internas:

Função atualizar_localidades

Esta função é chamada quando o utilizador seleciona um distrito.

```
def atualizar_localidades(localidade_selecionada):  
    if localidade_selecionada == "Selecionar":  
        # Limpa o menu de localidades e define a opção padrão  
        menu_localidade.configure(values=["Selecionar"])  
        valor_localidade.set("Selecionar")  
        return  
  
    # Procura o ID do distrito a partir do nome selecionado  
    id_distrito = next(  
        (id_d for id_d, nome in API.distritos_por_id.items() if nome == localidade_selecionada),  
        None  
    )  
  
    # Se não encontrar um ID válido, define novamente os valores padrão  
    if id_distrito is None:  
        menu_localidade.configure(values=["Selecionar"])  
        valor_localidade.set("Selecionar")  
        return  
  
    # Filtra as localidades que pertencem ao distrito selecionado  
    locais_filtrados = API.filtrar_localidades_por_distrito(localidades, id_distrito)  
  
    # Extrai apenas os nomes das localidades  
    nomes_locais = [loc['local'] for loc in locais_filtrados]  
  
    # Atualiza o dropdown de localidades com os nomes filtrados  
    menu_localidade.configure(values=["Selecionar"] + nomes_locais)  
    valor_localidade.set("Selecionar") # Define o valor padrão como "Selecionar"
```

Com base no nome escolhido, é obtido o respetivo ID, que é usado para filtrar e listar as localidades pertencentes a esse distrito. O menu de localidades é então atualizado com estas opções.

Função mostrar_previsao

Esta função é chamada quando o utilizador escolhe uma localidade. Começa por limpar previsões da interface.

Em seguida, verifica se o utilizador selecionou um distrito e uma localidade válidos. Se sim, obtém o ID do distrito filtra as localidades e encontra a localidade correspondente. Com o ID global da localidade, obtém a previsão a previsão meteorológica para o número de dias especificado.

Distrito:

Aveiro

Localidade:

Águeda

Dias de previsão:

1

Distrito:

Aveiro

Localidade:

Águeda

Dias de previsão:

3

Data: 02-06-2025
Temp. Máx: 24.5°C
Temp. Min: 13.6°C
Chuva: 0.0%



Data: 02-06-2025
Temp. Máx: 24.5°C
Temp. Min: 13.6°C
Chuva: 0.0%



Data: 03-06-2025
Temp. Máx: 21.1°C
Temp. Min: 12.0°C
Chuva: 4.0%



Data: 04-06-2025
Temp. Máx: 22.0°C
Temp. Min: 12.4°C
Chuva: 40.0%



Distrito:

Aveiro

Localidade:

Águeda

Dias de previsão:

5

Data: 02-06-2025
Temp. Máx: 24.5°C
Temp. Min: 13.6°C
Chuva: 0.0%



Data: 03-06-2025
Temp. Máx: 21.1°C
Temp. Min: 12.0°C
Chuva: 4.0%



Data: 04-06-2025
Temp. Máx: 22.0°C
Temp. Min: 12.4°C
Chuva: 40.0%



Data: 05-06-2025
Temp. Máx: 23.2°C
Temp. Min: 10.9°C
Chuva: 0.0%



Data: 06-06-2025
Temp. Máx: 24.3°C
Temp. Min: 11.5°C
Chuva: 4.0%



Os dados são convertidos e validados, incluindo as temperaturas, a precipitação e o tipo de tempo. Para cada dia da previsão, cria um registo no histórico da base de dados, tenta carregar o ícone correspondente ao tipo de tempo, e apresenta a informação formatada num quadro com texto e imagem.

Por fim, a função principal configura os menus suspensos para que, ao serem alterados, ativem as funções apropriadas – ou seja, quando se escolhe um distrito, as localidades são atualizadas; ao escolher uma localidade ou alterar o número de dias, a previsão meteorológica é exibida.

Distrito:

Faro

Localidade:

Portimão

Dias de previsão:

1

Distrito:

Faro

Localidade:

Portimão

Dias de previsão:

3

Data: 02-06-2025

Temp. Máx: 26.7°C

Temp. Min: 15.8°C

Chuva: 0.0%



Data: 02-06-2025

Temp. Máx: 26.7°C

Temp. Min: 15.8°C

Chuva: 0.0%



Data: 03-06-2025

Temp. Máx: 23.5°C

Temp. Min: 14.4°C

Chuva: 0.0%



Data: 04-06-2025

Temp. Máx: 26.3°C

Temp. Min: 13.9°C

Chuva: 0.0%



Distrito:

Faro

Localidade:

Portimão

Dias de previsão:

5

Data: 02-06-2025

Temp. Máx: 26.7°C

Temp. Min: 15.8°C

Chuva: 0.0%



Data: 03-06-2025

Temp. Máx: 23.5°C

Temp. Min: 14.4°C

Chuva: 0.0%



Data: 04-06-2025

Temp. Máx: 26.3°C

Temp. Min: 13.9°C

Chuva: 0.0%



Data: 05-06-2025

Temp. Máx: 28.8°C

Temp. Min: 13.4°C

Chuva: 0.0%



Data: 06-06-2025

Temp. Máx: 31.8°C

Temp. Min: 14.9°C

Chuva: 0.0%



41

Função atualizar_previsao_localidade

Tem como objetivo obter e apresentar a previsão meteorológica de uma determinada localidade, nomeadamente a cidade atual do utilizador.

Localidade:

Dias:

Localidade:

Dias:

Data: 02-06-2025

Temp. Máx: 27.7°C

Temp. Min: 14.0°C

Chuva: 2.0%



Data: 03-06-2025

Temp. Máx: 24.2°C

Temp. Min: 12.7°C

Chuva: 4.0%



Data: 04-06-2025

Temp. Máx: 23.0°C

Temp. Min: 11.1°C

Chuva: 40.0%



Localidade:

Dias:

Data: 02-06-2025

Temp. Máx: 27.7°C

Temp. Min: 14.0°C

Chuva: 2.0%



Data: 03-06-2025

Temp. Máx: 24.2°C

Temp. Min: 12.7°C

Chuva: 4.0%



Data: 04-06-2025

Temp. Máx: 23.0°C

Temp. Min: 11.1°C

Chuva: 40.0%



Data: 05-06-2025

Temp. Máx: 25.9°C

Temp. Min: 9.8°C

Chuva: 0.0%



Data: 06-06-2025

Temp. Máx: 27.4°C

Temp. Min: 10.4°C

Chuva: 2.0%



Inicialmente, a função verifica se foi selecionado um número válido de dias para previsão.

Caso contrário, termina sem fazer alterações. Em seguida, obtém o identificador global de localidade através da API, utilizando o nome da cidade. Se não for possível obter esse identificador, é apresentada uma mensagem de erro ao utilizado. Caso o identificador seja obtido com sucesso a função solicita à API a previsão do tempo agregada para os dias pretendidos.

Antes de exibir a nova previsão, a função limpa o conteúdo gráfico anterior, evitando a sobreposição de informação. Depois, tenta carregar um ficheiro local em formato JSON (tipo_de_tempo.json) que contém as descrições dos tipos de tempo em português. Para cada dia da previsão (até ao número de dias selecionado), a função extrai e converte os valores de temperatura mínima, temperatura máxima, probabilidade de precipitação, data e tipo de tempo.

Com estes dados, regista cada previsão no histórico do utilizador. Por fim, cria graficamente um bloco (frame) que apresenta a data, as temperaturas e a precipitação, assim como o ícone correspondente ao tipo de tempo, se disponível.

Caso ocorra algum erro durante o processo, são apresentadas mensagens de aviso ao utilizador.

```
messagebox.showerror("Erro!",  
| | | | | "Erro na obtenção da previsão. Verifique a ligação à internet ou tente novamente mais tarde.")
```

Função calcular_media_registos

Serve para calcular a média de valores meteorológicos, nomeadamente a temperatura mínima, a temperatura máxima e a precipitação, com base num conjunto de registos.

```
def calcular_media_registos(registos):  
    # Inicializa as somas e o contador de registos  
    soma_min = 0  
    soma_max = 0  
    soma_precip = 0  
    contagem = 0  
  
    # Espera-se que cada tupla contenha (temp_min, temp_max, precip)  
    for temp_min, temp_max, precip in registos:  
        soma_min += temp_min    # Soma das temperaturas mínimas  
        soma_max += temp_max    # Soma das temperaturas máximas  
        soma_precip += precip   # Soma da precipitação  
        contagem += 1          # Conta o número de registos válidos  
  
    # Se não houver registos, retorna None para evitar divisão por zero  
    if contagem == 0:  
        return None  
  
    # Retorna um dicionário com as médias calculadas  
    return {  
        "media_min": soma_min / contagem,    # Média da temperatura mínima  
        "media_max": soma_max / contagem,    # Média da temperatura máxima  
        "media_chuva": soma_precip / contagem # Média da precipitação  
    }
```

A função começa por inicializar variáveis de soma para cada um destes parâmetros, bem como um contador. Em seguida, percorre os registos, soma e conta o número total de entradas válidas. Se não existir registos, a função vai retornar None, evitando divisões por zero. Caso contrário, calcula a média de cada parâmetro e devolve um dicionário com os respetivos valores.

Função `ver_media`

Tem como finalidade apresentar ao utilizador as médias meteorológicas dos últimos registos armazenados no histórico, para um determinado tipo de procura, que pode ser “Localidade” ou “Destino”.

Inicialmente a função obtém esses registos do histórico com base no identificador do utilizador e no tipo de procura especificado. De seguida, limpa todos os elementos gráficos anteriores existentes no `frame_destino`.

Se não existirem registos para o tipo de procura indicado, é mostrado um texto a informar que não há dados suficientes para calcular médias. Caso contrário, são calculadas as médias de temperatura mínima, temperatura máxima e precipitação através da função `calcular_media_registos`.

```
medias = calcular_media_registos(registos) # Calcula as médias a partir dos registos
```

Se esta devolve `None` (ou seja, se não for possível calcular médias), é igualmente apresentada uma mensagem ao utilizador a indicar a ausência de dados válidos.

```
if not medias:
    texto = f"Sem dados de {tipo_procura} para os últimos 30 registos."
else:
    # Cria o texto com os dados médios formatados
    texto = (
        f"📊 Média dos Últimos 30 Dias ({tipo_procura})\n"
        f"🌡️ Temp. Mínima Média: {medias['media_min']:.1f}°C\n"
        f"🔥 Temp. Máxima Média: {medias['media_max']:.1f}°C\n"
        f"☔ Precipitação Média: {medias['media_chuva']:.1f}%"
    )
```

Caso contrário, é contruído um texto formatado com as médias calculadas e este é mostrado num label inserido dentro do `frame_destino`.

Na interface gráfica da aplicação, foram criados dois botões principais que permitem ao utilizador adicionar esta função: uma para consultar a média meteorológica da localidade atual (`botao_vermed_met_localidade`) e outro para o destino escolhido (`botao_vermed_met_destino`).

```
botao_vermed_met_localidade = ctk.CTkButton(
    janela_APP,
    text="Ver Média Meteorologia (Localidade)",
    command=lambda: ver_media("Localidade", frame_combobox_localidade))
botao_vermed_met_localidade.place(relx=0.225, rely=0.95, anchor="center")

botao_vermed_met_destino = ctk.CTkButton(
    janela_APP,
    text="Ver Média Meteorologia (Destino)",
    command=lambda: ver_media("Destino", frame_combobox_destino))
botao_vermed_met_destino.place(relx=0.725, rely=0.95, anchor="center")
```

Estes botões invocam a função `ver_media` com o argumento apropriado quando clicados.

Ver Média Meteorologia (Localidade)

 Média dos Últimos 30 Dias (Localidade)
 Temp. Mínima Média: 12.2°C
 Temp. Máxima Média: 25.6°C
 Precipitação Média: 10.7%

Ver Média Meteorologia (Destino)

 Média dos Últimos 30 Dias (Destino)
 Temp. Mínima Média: 13.5°C
 Temp. Máxima Média: 25.0°C
 Precipitação Média: 5.0%

Além disso, é configurado o menu de seleção de dias para previsão (`menu_dias_localidade`), que permite ao utilizador escolher 1,3 e 5 dias.

```
menu_dias_localidade = ctk.CTkOptionMenu(janela_APP, values=["1", "3", "5"],
                                         variable=valor_dias_localidade, width=124, height=30,
                                         font=("Arial", 15), fg_color="white", button_color="white",
                                         text_color="black", dropdown_font=("Arial", 15),
                                         command=atualizar_previsao_localidade)
menu_dias_localidade.place(relx=0.155, rely=0.23, anchor="center")
```

Quando o utilizador faz uma nova seleção neste menu, é chamada automaticamente a função `atualizar_previsao_localidade`, que atualiza a previsão apresentada com base na sua nova escolha.

Por fim, o código define elementos visuais como rótulos (labels) para indicar a localidade, os dias a prever e também um botão com imagem associado à função de logout. A imagem é carregada de um ficheiro, e, se ocorrer algum erro no processo, é apresentada uma mensagem de erro ao utilizador.



Conclusão

O presente projeto teve como finalidade a conceção e desenvolvimento de uma aplicação gráfica, funcional e interativa, dedicada à consulta de previsões meteorológicas. Através da utilização da linguagem de programação Python, aliada à biblioteca CustomTkinter para construção da interface gráfica, foi possível criar uma solução eficiente que integra dados provenientes de uma API pública, garantindo ao utilizador previsões atualizadas e relevantes.

A aplicação contempla funcionalidades como a seleção de localidade e número de dias de previsão, a apresentação de dados meteorológicos com elementos visuais (ícones), bem como o registo automático de consultas realizadas, armazenadas localmente numa base de dados. Adicionalmente, foi desenvolvida a funcionalidade de cálculo de médias meteorológicas com base nos últimos registos, permitindo uma análise estatística simples, mas útil para o utilizador.

Ao longo do desenvolvimento foram adotadas boas práticas de programação orientada a objetos, organização modular do código e tratamento de exceções, com o intuito de garantir a robustez, legibilidade e manutenção futura da aplicação. A arquitetura da aplicação permite ainda uma fácil escalabilidade, possibilitando a integração de novas funcionalidades ou fontes de dados, caso tal venha a ser necessário.

De forma global, este projeto constituiu uma oportunidade para aplicar e consolidar conhecimentos nas áreas de desenvolvimento de interfaces gráficas, integração com serviços externos via API, gestão de bases de dados locais (SQLite) e manipulação de ficheiros. A solução desenvolvida demonstra, assim, não só a viabilidade técnica do conceito, mas também o seu potencial de aplicação prática, tanto em contextos pessoais como educativos.

Caso contrário, faz login normal com o nome /UID e senha.

FIM!